601.433/633 Introduction to Algorithms **Fall 2026**
Homework #4 **Due:** October 15, 2026, 11:59pm

Remember: you may work in groups, but must write up your solution entirely on your own. Solutions must by typeset (LaTeX preferred but not required). Collaboration is limited to discussing the problems – you may not look at, compare, reuse, etc. any text from anyone else in the class. You may use the internet to look up formulas, definitions, etc., but may not simply look up the answers online.

Please include proofs with all of your answers, unless stated otherwise.

1 Hashing (50 points)

We say that H is a 2-*universal hash family* if $\mathbf{Pr}_{h\sim H}[h(x) = a \wedge h(y) = b] \leq 1/M^2$ for all $x, y \in U$ with $x \neq y$ and $a, b \in \{0, 1, \ldots, M-1\}$.

(a) (25 points) Prove that any 2-universal hash family is also a universal hash family.

(b) (25 points) Prove that for every u and b with $u > b \geq 1$ there is some universal hash family from U to $\{0, 1, \ldots, M-1\}$ (with $|U| = 2^u$ and $M = 2^b$) which is *not* a 2-universal hash family. Hint: think about the constructions from class and the textbook.

2 Union-Find (50 points)

In class we proved that if we use trees to represent disjoint sets and use both union-by-rank and path compression, then the amortized cost of any operation (Make-Set, Find, or Union) is only $O(\log^* n)$. In this question we'll analyze what happens if we change our data structure in two ways: we do not use path compression, and we do union-by-cardinality rather than union-by-rank.

Slightly more formally, we change our data structure as follows. As before, every node has three values: an element, a parent pointer, and a value. But the value isn't a rank, but is rather the number of nodes in the subtree rooted at it, i.e., the *cardinality* of the subtree. So the root of any tree stores the cardinality of the set represented by the tree. The basic operations work as follows:

- Make-Set(x) simply returns a single node with the element x, where the parent pointer points to itself and the initial cardinality is 1.
- Find(x) follows the parent pointer from x recursively until it ends up at the root (which it knows since only the root will have its parent pointer point to itself), and then it returns the element at this root.
- Union(x, y) first does Find(x) to find the root r_x of the tree containing x and Find(y) to find the root r_y of the tree containing y. If $(r_x \to \textit{cardinality}) \geq (r_y \to \textit{cardinality})$, then we set the parent of r_y to be r_x and set $(r_x \to \textit{cardinality}) = (r_x \to \textit{cardinality}) + (r_y \to \textit{cardinality})$. Similarly, if $(r_y \to \textit{cardinality}) > (r_x \to \textit{cardinality})$, then we set the parent of r_x to be r_y and set $(r_y \to \textit{cardinality}) = (r_x \to \textit{cardinality}) + (r_y \to \textit{cardinality})$.

(a) (25 points) Prove that the worst-case running time of any operation is $O(\log n)$, where n is the number of Make-Set operations (i.e., the number of elements). Hint: can you bound the depth/height of a tree by its cardinality?

(b) (25 points) We now want to provide a matching lower bound not just on the worst-case running time, but even on the amortized running time. So suppose that we first do n Make-Set operations (so there are n elements). Give a sequence of $O(n)$ Union operations which cumulatively take $\Omega(n \log n)$ time (implying that the amortized cost of an operation is $\Omega(\log n)$). Each union operation should be a union of two distinct sets, i.e., you should never call Union(x,y) on elements x and y that are in the same set.